

Boolean Algebra

George Boole (1815-1864) English mathematician and philosopher.

- Defined on the set $B = \{0,1\}$
- Binary operators: AND, OR $\{ \cdot, + \}$
- Unary operation: Complement (NOT) $\{ ' \}$
Another symbol for the NOT operator: $\bar{a}$

$a \cdot b$			$a + b$			Complement	
b \ a	0	1	b \ a	0	1	a	a'
0	0	0	0	0	1	0	1
1	0	1	1	1	1	1	0

Axioms (Laws):

Under assumption $a, b \in B$

- Closure: $a + b \in B$ $a \cdot b \in B$
- Commutative: $a + b = b + a$ $a \cdot b = b \cdot a$
- Associative: $a + (b + c) = (a + b) + c$ $a \cdot (b \cdot c) = (a \cdot b) \cdot c$
- Identity: $a + 0 = a$ $a \cdot 1 = a$
- Distributive: $a + (b \cdot c) = (a + b) \cdot (a + c)$ $a \cdot (b + c) = (a \cdot b) + (a \cdot c)$
- Inverse: $a + \bar{a} = 1$ $a \cdot \bar{a} = 0$

Properties and Theorems:

These properties and theorems are derived from the operations and axioms of the Boolean algebra.

They can be proven using the axioms.

- Annihilator (or Dominance):
 $a + 1 = 1$ $a \cdot 0 = 0$
- Involution: $(a')' = a$ or $\bar{\bar{a}} = a$
- Idempotency:
 $a + a + \dots + a = a$ $a \cdot a \cdot \dots \cdot a = a$
- Absorption: (Proof in 2.4)
 $a + a \cdot b = a$ $a \cdot (a + b) = a$
- De Morgan Theorem: *Augustus De Morgan (1806 - 1871)*
 $(a + b)' = a' \cdot b'$ $(a \cdot b)' = a' + b'$
- General De Morgan Theorem:
 $f'(X_1, X_2, \dots, X_n, 0, 1, +, \cdot) = f(X_1', X_2', \dots, X_n', 1, 0, \cdot, +)$
 - It establishes relations between binary operations (AND, OR): $\cdot$ and $+$

6. The Duality principle

A dual of a logical expression is obtained by replacing, $\cdot$ by $+$, $+$ by $\cdot$, 0 by 1, and 1 by 0, without changing the variables.

$$a + b + 0 \dots \Leftrightarrow a \cdot b \cdot 1 \dots$$

Duals of all proven theorems are also valid.

If we can prove a theorem than we know that its dual is also true.

Example:

Absorption:

If we can prove the theorem $a + a \cdot b = a$, than its dual $a \cdot (a+b) = a$ is also true.

Note that in previous slides axioms and theorems are presented with their duals.

General Duality Principle:

$$f(X_1, X_2, \dots, X_n, 0, 1, +, \cdot) \Leftrightarrow f^D(X_1, X_2, \dots, X_n, 1, 0, \cdot, +)$$

- It is different from the De Morgan Theorem.
 - It establishes relation between proofs of theorems.
 - It is not used to convert one expression into another. Duals are not equal.

Precedence Order Of Operators:

From higher to lower precedence:

1. Parenthesis,
2. Complement (Negation),
3. AND,
4. OR

Proving the theorems:

a) With Axioms

Example:

Theorem: $X \cdot Y + X \cdot Y' = X$

Proof:

Distributive	$X \cdot Y + X \cdot Y'$	$= X \cdot (Y + Y')$
Complement	$X \cdot (Y + Y')$	$= X \cdot (1)$
Identity	$X \cdot (1)$	$= X \checkmark$

Example:

Theorem: $X + X \cdot Y = X$ (*Absorption*)

Proof:

Identity	$X + X \cdot Y$	$= X \cdot 1 + X \cdot Y$
Distributive	$X \cdot 1 + X \cdot Y$	$= X \cdot (1 + Y)$
Annihilator	$X \cdot (1 + Y)$	$= X \cdot (1)$
Identity	$X \cdot (1)$	$= X \checkmark$

Proving the theorems: **b) With the truth table**

Note that to denote the negation (NOT) the notation $\overline{A}$ can also be used.

De Morgan:

$$\overline{(X + Y)} = \overline{X} \cdot \overline{Y}$$

X	Y	$\overline{X}$	$\overline{Y}$	$\overline{(X + Y)}$	$\overline{X} \cdot \overline{Y}$
0	0	1	1	1	1
0	1	1	0	0	0
1	0	0	1	0	0
1	1	0	0	0	0

$$\overline{(X \cdot Y)} = \overline{X} + \overline{Y}$$

X	Y	$\overline{X}$	$\overline{Y}$	$\overline{(X \cdot Y)}$	$\overline{X} + \overline{Y}$
0	0	1	1	1	1
0	1	1	0	1	1
1	0	0	1	1	1
1	1	0	0	0	0

Although truth tables may contain many rows, they can be filled and compared by computer programs in very short times.

Theorems are also used to minimize the logical expressions:

Example:

$$\begin{aligned}
 Z &= A'BC + AB'C + ABC' + ABC \\
 &= A'BC + AB'C + ABC' + ABC + ABC \\
 &= A'BC + ABC + AB'C + ABC' + ABC \\
 &= (A' + A)BC + AB'C + ABC' + ABC \\
 &= (1)BC + AB'C + ABC' + ABC \\
 &= BC + AB'C + ABC' + ABC + ABC \\
 &= BC + AB'C + ABC + ABC' + ABC \\
 &= BC + A(B' + B)C + ABC' + ABC \\
 &= BC + A(1)C + ABC' + ABC \\
 &= BC + AC + AB(C' + C) \\
 &= BC + AC + AB(1) \\
 &= BC + AC + AB
 \end{aligned}$$

Logical (Boolean) Expressions

A logical expression, is a finite combination of variables, constants and operators that are well-formed according to the rules.

It is represented as $E(X)$, where $X = (x_1, x_2, \dots, x_n)$ and each $x_i \in \{0,1\}$.

If E_1 and E_2 are logical expressions then, E_1' , E_2' , $E_1 + E_2$, $E_1 \cdot E_2$ and all possible combinations are also logical expressions.

Normal Forms of Logical expressions:

Each logical expression can be written in two special forms.

1. Disjunctive normal form (DNF): $\Sigma\Pi$

Logical sum of logical products (SOP). OR of ANDs.

Example: $bc' + ad + a'b$

The OR operation is also called the *logical disjunction*.

2. Conjunctive normal form (CNF): $\Pi\Sigma$

Logical product of logical sums (POS). AND of ORs.

Example: $(a+b+c')(a+d)(a'+b)$

The AND operation is also called the *logical conjunction*.

Any logical expression can be written in CNF (POS form) and DNF (SOP form).

Any expression in CNF can be converted to DNF and vice versa.

Value of a logical expression:

An expression $E(X)$, generates for each combination of the input vector $X = (x_1, \dots, x_n)$, a value from the set of $B = \{0,1\}$.

These values constitute the truth table of the expression.

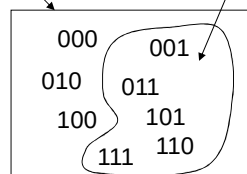
Example: $E(X) = x_1x_2 + x_3$

Truth table of the expression

$X =$	x_1	x_2	x_3	$E(X)$
	0	0	0	0
	0	0	1	1
	0	1	0	0
	0	1	1	1
	1	0	0	0
	1	0	1	1
	1	1	0	1
	1	1	1	1

Set of all input combinations (X)

Combinations for which $E(X)$ generates '1' (covers)



"Order Relation" between expressions:

To explain some properties of the logical expressions the following order relation can be used.

An order relation between elements of set $B=\{0,1\}$: $0 < 1$

Read as "0 precedes 1" or "0 is smaller than 1".

According to this, an order relation between X vectors can be defined as follows:

If all elements of X1 precede (are smaller than) elements in the same order of vector X2 then $X1 \leq X2$.

Example:

$X1=1001$, $X2 = 1101$

$X1 \leq X2$.

The order relation may not exist between all vectors.

Example: $X1=0011$, $X2 = 1001$

The order relation is not valid between X1 and X2.

Order relation between expressions:

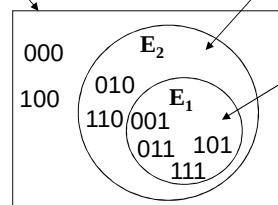
$E_1(X) \leq E_2(X)$ denotes that, values of E_1 generated by combinations of input X are always smaller than (or equal to) all values of E_2 generated by the same input combinations.

Example :

x_1	x_2	x_3	$E_1(X)$	$E_2(X)$
0	0	0	0	0
0	0	1	1	1
0	1	0	0	1
0	1	1	1	1
1	0	0	0	0
1	0	1	1	1
1	1	0	0	1
1	1	1	1	1

Set of all input combinations (X)

Combinations for which $E_2(X)$ generates '1' (covers)



Combinations for which $E_1(X)$ generates '1' (covers)

For each input combination, where $E_1(X)$ generates 1, $E_2(X)$ also generates 1. (This is a special case.)

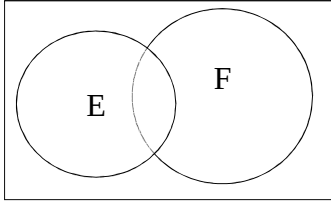
If $E_1(X) \leq E_2(X)$:

1. $E_1(X) + E_2(X) = E_2(X)$
2. $E_1(X) \cdot E_2(X) = E_1(X)$

If $E_1(X) \leq E_2(X)$

$E_1(X)$ implies $E_2(X)$, $E_1(X) \Rightarrow E_2(X)$,
 $E_2(X)$ covers $E_1(X)$.

The order relation ($\leq$) may not be valid between all expressions.



IF E and F are two logical expressions the following inequalities are always true:

$$E \cdot F \leq E \leq E + F$$

$$E \cdot F \leq F \leq E + F$$

Absorption Properties of expressions:

$$E + E \cdot F = E$$

dual

$$E(E + F) = E$$

$$\text{Proof: } E(E + F) = EE + EF = E + EF = E(1 + F) = E$$

$$E + E' \cdot F = E + F$$

dual

$$E(E' + F) = E \cdot F$$

$$\text{Proof: } E + E' \cdot F = (E + E')(E + F) = 1(E + F) = E + F$$

These properties are used to minimize (simplify) logical expressions.

Example:

$E(a,b,c,d) = abc'$, $F(a,b,c,d) = bd$ There is not an order relation between E and F.

$$E \cdot F = abc'd \quad E + F = abc' + bd$$

a b c d	E	F	$E \cdot F$	$E + F$
0 0 0 0	0	0	0	0
0 0 0 1	0	0	0	0
0 0 1 0	0	0	0	0
0 0 1 1	0	0	0	0
0 1 0 0	0	0	0	0
0 1 0 1	0	1	0	1
0 1 1 0	0	0	0	0
0 1 1 1	0	1	0	1
1 0 0 0	0	0	0	0
1 0 0 1	0	0	0	0
1 0 1 0	0	0	0	0
1 0 1 1	0	0	0	0
1 1 0 0	1	0	0	1
1 1 0 1	1	1	1	1
1 1 1 0	0	0	0	0
1 1 1 1	0	1	0	1

$$E \cdot F < E \quad \text{and} \quad E \cdot F < F.$$

Therefore

$$E \cdot F + E = E$$

$$abc'd + abc' = abc'$$

and

$$E \cdot F + F = F$$

$$abc'd + bd = bd$$

$$E < E + F \quad \text{and} \quad F < E + F.$$

Therefore

$$E \cdot (E + F) = E$$

$$abc' (abc' + bd) = abc'$$

and

$$F \cdot (E + F) = F$$

$$bd (abc' + bd) = bd$$

The Consensus Theorem

Assume that E_1 and E_2 are two expressions that do not include the literal x_1 :
 $E_1(x_2, \dots, x_n)$ and $E_2(x_2, \dots, x_m)$

We can create a new expression by multiplying one term by x_1 and the other one by the complement of x_1 .

$$E = x_1 E_1 + x_1' E_2$$

Here, x_1 is called a **biform** variable, because it occurs both positively (as itself: x_1) and negatively (as complement: $\bar{x}_1$) in the expression.

Examples: $x_1(x_2+x_3') + x_1'(x_3+x_4)$,
 $x_1x_2x_3 + x_1'x_4x_5$

• Given a pair of product terms that include a biform variable, **the consensus term** is formed by multiplying two original terms together, leaving out the selected variable and its complement.

• E_1E_2 is the **consensus term** of $x_1E_1 + x_1'E_2$ with respect to the variable x .

Example: Consensus of $abc + a'cd$ (respect to a) is $bcd = bcd$

Theorem: The consensus term is redundant and can be eliminated.

$$x_1E_1 + x_1'E_2 + E_1E_2 = x_1E_1 + x_1'E_2$$

The Consensus theorem (dual)

According to the duality principle the consensus theorem is also valid for expressions written in product of sums (POS) form.

Assume that E_1 and E_2 are two expressions, which do not include the literal x_1 :
 $E_1(x_2, \dots, x_n)$ and $E_2(x_2, \dots, x_m)$

We can create a new expression by adding x_1 to one term and the complement of x_1 to the other one.

$$E = (x_1 + E_1)(x_1' + E_2)$$

Here, x_1 is a **biform** variable.

Examples: $(x_1+x_2+x_3')(x_1'+x_3+x_4)$,
 $(x_1+x_2x_3')(x_1'+x_3x_4)$

• Given a pair of sums that include a biform variable, **the consensus term** is formed by adding two original terms together, leaving out the selected variable and its complement.

• $E_1 + E_2$ is the **consensus term** of $(x_1+E_1)(x_1'+E_2)$ with respect to the variable x .

Example: Consensus of $(a+b+c)(a'+c+d)$ is: $b+c+c+d = b+c+d$

Theorem: The consensus term is redundant and can be eliminated.

$$(x_1+E_1)(x_1'+E_2)(E_1+E_2) = (x_1+E_1)(x_1'+E_2)$$

Example: Minimization of a logical expression using the consensus theorem

$$\begin{aligned}
 F(A, B, C) &= A'B'C + A'BC + AB'C + ABC + ABC' \\
 &= A'B'C + A'BC + AB'C + ABC + \cancel{ABC'} + \cancel{AB} \quad \text{Consensus (respect to C) is added} \\
 &= A'B'C + A'BC + AB'C + \cancel{ABC} + \cancel{ABC'} + AB \quad \text{Absorption} \\
 &= A'B'C + A'BC + AB'C + AB \\
 &= A'B'C + A'BC + \cancel{A'C} + \cancel{AB'C} + AB \quad \text{Consensus (respect to B) is added} \\
 &= \cancel{A'B'C} + \cancel{A'BC} + A'C + AB'C + AB \quad \text{Absorption} \\
 &= A'C + AB'C + AB \\
 &= A'C + \cancel{AB'C} + AB + \cancel{AC} \quad \text{Consensus (B) is added} \\
 &= A'C + AB + AC \quad \text{Absorption} \\
 &= \cancel{A'C} + AB + \cancel{AC} + \cancel{C} \quad \text{Consensus (A) is added} \\
 &= AB + C \quad \text{Absorption}
 \end{aligned}$$

Logical (Boolean) Functions

Logical functions are defined on the input set B^n (vectors with n binary variables). There are 3 types of logical functions:

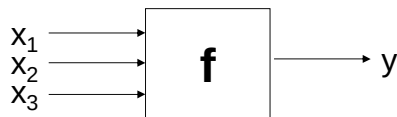
1. Simple (basic) functions: Multiple inputs, single output

$$y = f(X): B^n \rightarrow B$$

$$\forall X^0 \in B^n; \exists! y^0 \in B; y = f(X)$$

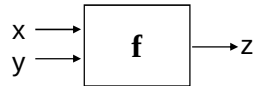
Example:

x_1	x_2	x_3	y
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1



There are $2^{(2^n)}$ possible basic logical functions for n binary variables (inputs).

There are 16 possible basic logical functions for 2 binary variables (inputs).



The truth table of 16 possible basic logical functions for 2 binary variables (F0–F15):

Inputs		Functions															
X	Y	F0	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10	F11	F12	F13	F14	F15
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

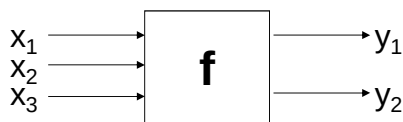
0 $\swarrow$ $X \text{ AND } Y$ X $\swarrow$ Y $\swarrow$ $X \text{ XOR } Y$ $\swarrow$ $X \text{ OR } Y$ $\swarrow$ $X \text{ NOR } Y$ $\swarrow$ $X = Y$ $\swarrow$ Y' $\swarrow$ X' $\swarrow$ $X \text{ NAND } Y$ $\swarrow$ 1
 (X OR Y)' (X AND Y)'

2. General functions: Multiple inputs, multiple outputs

$$Y = f(X): B^n \rightarrow B^m, \quad X = (x_1, \dots, x_n), \quad Y = (y_1, \dots, y_m),$$

Example:

x_1	x_2	x_3	y_1	y_2
0	0	0	1	1
0	0	1	1	0
0	1	0	0	0
0	1	1	0	0
1	0	0	1	1
1	0	1	0	1
1	1	0	0	1
1	1	1	1	0

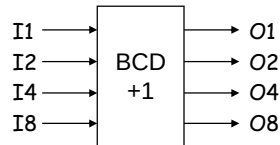


3. Incompletely specified functions: They have unspecified outputs for some input combinations.

The input-sequence of the function that the designer does not care about is called **don't-care** condition.

A don't-care input condition usually never happens, or differences in that don't result in any changes to the output.

Example: A function that increments BCD numbers



I8	I4	I2	I1	O8	O4	O2	O1
0	0	0	0	0	0	0	1
0	0	0	1	0	0	1	0
0	0	1	0	0	0	1	1
0	0	1	1	0	1	0	0
0	1	0	0	0	1	0	1
0	1	0	1	0	1	1	0
0	1	1	0	0	1	1	1
0	1	1	1	1	0	0	0
1	0	0	0	1	0	0	1
1	0	0	1	0	0	0	0
1	0	1	0	X	X	X	X
1	0	1	1	X	X	X	X
1	1	0	0	X	X	X	X
1	1	0	1	X	X	X	X
1	1	1	0	X	X	X	X
1	1	1	1	X	X	X	X

For these input combinations the output values of the circuit (function) are not specified.

They can not happen.

Don't-care conditions are presented by an X or Φ .

Representation of Logical (Boolean) Functions

There are different ways to represent (express) the same logical function.

While designing logical circuits the proper representation method can be used.

Representation with truth table:

Output values for all possible input combinations (variables) are written in a table.

The input columns are usually constructed in the order of binary counting.

Numbered (Indexed) Representation:

Input variables are encoded as binary numbers.

We can assign a decimal number for each input combination according to its binary value.

To represent the function we can list the number of input combinations for which the function generates the logical value "1" (or logical "0" or " Φ ").

Example: Representation of a completely specified basic logical function:

Num	Input $x_1 \ x_2$	Output y
0	0 0	1
1	0 1	0
2	1 0	1
3	1 1	0

$$y = f(x_1, x_2) = \cup_1(0, 2)$$

$\cup$ denotes "union" or "set of".

$\cup_1$ denotes "set of 1-generating points".

The order of the variables (x_1, x_2) is important.
It must be the same as in the truth table.

Otherwise the numbers of the combinations will change.

The same function; only
the order of the variables
(x_2, x_1) is changed.

Num	Input $x_2 \ x_1$	Output y
0	0 0	1
1	0 1	1
2	1 0	0
3	1 1	0

$$y = f(x_2, x_1) = \cup_1(0, 1)$$

The same function can be represented with "0"-generating combinations.

$$y = f(x_1, x_2) = \cup_0(1, 3)$$

Example: Representation of a completely specified general logical function :

The numbered representation is applied to all outputs.

Num	$x_1 \ x_2$	$y_1 \ y_2$
0	0 0	1 1
1	0 1	0 1
2	1 0	1 0
3	1 1	0 0

$$y_1 = f(x_1, x_2) = \cup_1(0, 2)$$

$$y_2 = f(x_1, x_2) = \cup_1(0, 1)$$

Representation of the same function with "0"-generating combinations:

$$y_1 = f(x_1, x_2) = \cup_0(1, 3)$$

$$y_2 = f(x_1, x_2) = \cup_0(2, 3)$$

Example: Representation of an incompletely general logical function:

In this case writing only 1-generating or 0-generating input combinations is not sufficient.

At least two different groups must be presented.

No	x_1	x_2	y_1	y_2
0	0	0	1	1
1	0	1	0	Φ
2	1	0	Φ	0
3	1	1	0	Φ

$$\begin{aligned}
 y_1 &= f(x_1, x_2) = \bigcup_1(0) + \bigcup_0(1, 3) \\
 \text{or} \quad y_1 &= f(x_1, x_2) = \bigcup_1(0) + \bigcup_\Phi(2) \\
 \text{or} \quad y_1 &= f(x_1, x_2) = \bigcup_0(1, 3) + \bigcup_\Phi(2) \\
 y_2 &= f(x_1, x_2) = \bigcup_1(0) + \bigcup_0(2) \\
 \text{or} \quad y_2 &= f(x_1, x_2) = \bigcup_1(0) + \bigcup_\Phi(1, 3) \\
 \text{or} \quad y_2 &= f(x_1, x_2) = \bigcup_0(2) + \bigcup_\Phi(1, 3)
 \end{aligned}$$

Graphical Representation

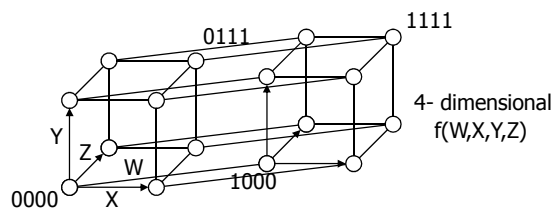
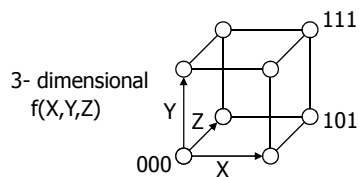
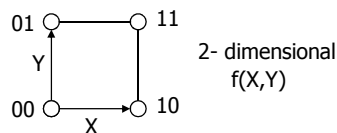
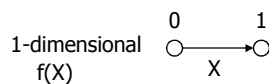
The input variables (combinations) of a logical function are elements (vectors) of the set B^n . Example: $B^3 = \{000, 001, \dots, 111\}$

These variables can be represented as vertices of an n-dimensional hyper cube.

Vertices that correspond to 1-generating combinations are colored (marked).

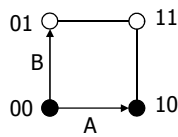
Number of inputs of the function defines the dimension of the cube.

n-bit input $\rightarrow$ n-dimensional cube

Boolean Cubes:

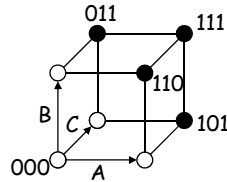
Example:

A	B	F
0	0	1
0	1	0
1	0	1
1	1	0



Example:

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1



With the increase in the variables (inputs) it gets harder to draw a Boolean cube. Therefore Boolean cubes are not practical to represent Boolean functions with many inputs.

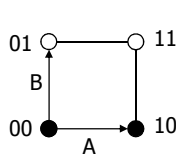
However, they make it easier to see some properties (especially the adjacent combinations) of the functions, and explain further topics.

Karnaugh Maps (Maurice Karnaugh (1924-), American physicist)

Karnaugh maps are used to represent and simplify Boolean functions.

The Boolean variables and results are transferred (generally from a truth table) into a table that is in a matrix form.

Num.	A	B	F
0	0	0	1
1	0	1	0
2	1	0	1
3	1	1	0



F	B	A
0	0	1
0	1	0
1	0	1
1	1	0

or

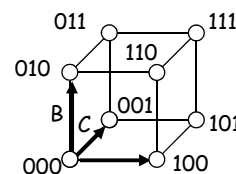
F	A	B
0	0	1
0	1	1
1	0	0
1	1	0

Cells of the table are ordered according to the principles of **Gray code**, where only one variable changes in between adjacent (horizontal and vertical) squares.

Format of the Karnaugh map of a function with 3 inputs:

		Gray Code				
		BC	00	01	11	10
Inputs	A	0	0	1	3	2
	1	4	5	7	6	

F	BC	000	001	011	010
A	0	0	0	1	0
1	1	0	1	1	1



Format of the Karnaugh map of a function with 4 inputs:

Gray Code

F AB		C D			
		00	01	11	10
A	00	0	1	3	2
	01	4	5	7	6
	11	12	13	15	14
	10	8	9	11	10

Gray Code

Example: The Karnaugh map of a function with 3 inputs :

Num	A	B	C	F
0	0	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	1

Karnaugh maps will be used in further lessons to simplify Boolean functions.

Algebraic Representation (Expressions) and Canonical Forms

A real-world digital problem can be specified by a truth table.

Example: Assume that input variable *A* represents that the door of a car is open, *B* represents that the key is inserted, then the truth table can specify the action *Z* to be taken, where *Z*=1 means that the alarm sounds.

Num.	A	B	Z
0	0	0	0
1	0	1	0
2	1	0	0
3	1	1	1

Truth tables of real-world digital problems are more complicated.

To handle digital problems and implement the solutions using logical gates, we need to obtain the **expressions** of the related Boolean function. $Z = f(A, B)$

Logical expressions of the Boolean functions can be obtained in **canonical forms** from their truth tables.

There are two types of canonical forms:

- 1st Canonical form: SOP ($\Sigma\Pi$) form. Example: $ab + cd$

It consists of a sum of products each correspond to a "1"-generating combination.

- 2nd Canonical form: POS ($\Pi\Sigma$) form. Example: $(a+b)(c+d)$

It consists of a product of sums each correspond to a "0"-generating combination.

1st Canonical Form: Sum of Products

- The 1st canonical form consists of a sum of product terms each correspond to a row in the truth table with the output "1".
- For a Boolean function of n variables, a product term in which each of the n variables appears once (in either its complemented or uncomplemented form) is called a **minterm**.
- For example a function with 3 variables (a, b, c) has 8 minterms:
 $a'b'c', a'b'c, a'bc', a'bc, ab'c', ab'c, abc', abc$
- Each minterm covers only one "true" row of the truth table.
- The 1st canonical form of a function is the sum of minterms.
- To find the minterms,
 - Locate all rows in the truth table where output is "1".
 - To generate the individual minterms, substitute variables (for example A, B or C) for ones (of the inputs) and complements of variables (A', B' , or C') for zeros in the truth table. (See the example in 2.30)
- A Boolean function may have many logical expressions. They produce the same result given the same inputs.
- But the expression in 1st canonical (standard) form is unique.

Example:

"True" (1) combinations: $F = 001 \quad 011 \quad 101 \quad 110 \quad 111$
 Sum of Minterms: $F = A'B'C + A'BC + AB'C + ABC' + ABC$

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

The 1st canonical form of the complement of the function ($\bar{F}$) can be obtained similarly by considering the "false" (0) rows:

A	B	C	F	F'
0	0	0	0	1
0	0	1	1	0
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	0

$$F' = A'B'C' + A'BC' + AB'C'$$

Indexing minterms:

We assign each minterm an index (number) based on binary encoding of the variables. For example, we assign the index 5 to the minterm $ab'c$ (101) and denote that minterm as m_5 .

A	B	C	Minterms
0	0	0	$A'B'C'$ m_0
0	0	1	$A'B'C$ m_1
0	1	0	$A'BC'$ m_2
0	1	1	$A'BC$ m_3
1	0	0	$AB'C'$ m_4
1	0	1	$AB'C$ m_5
1	1	0	ABC' m_6
1	1	1	ABC m_7

Expression of function F in (2.30) in 1st canonical form:

$$\begin{aligned} F(A, B, C) &= \Sigma m(1, 3, 5, 6, 7) \\ &= m_1 + m_3 + m_5 + m_6 + m_7 \\ &= A'B'C + A'BC + AB'C + ABC' + ABC \\ F &= \Sigma_{A, B, C} (1, 3, 5, 6, 7) \text{ (Sum of minterms)} \end{aligned}$$

Canonical forms are usually not the simplest (optimal) algebraic expression of the function.

The can be usually simplified.

Representation of minterms with 3 variables

Simplification:

$$\begin{aligned} F(A, B, C) &= A'B'C + A'BC + AB'C + ABC + ABC' \\ &= (A'B' + A'B + AB' + AB)C + ABC' \\ &= ((A' + A)(B' + B))C + ABC' \\ &= C + ABC' \\ &= ABC' + C \\ &= AB + C \end{aligned}$$

2nd Canonical Form: Product of Sums

- The 2nd canonical form consists of a product of sum terms each correspond to a row in the truth table with the output "0".
- For a Boolean function of n variables, a sum term in which each of the n variables appears once (in either its complemented or uncomplemented form) is called a **maxterm**.
- For example a function with 3 variables (a, b, c) has 8 maxterms:
 $a+b+c, a+b+c', a+b'+c, a+b'+c', a'+b+c, a'+b+c', a'+b'+c, a'+b'+c'$
- Each maxterm is "false" for exactly one combination of inputs.
- The 2nd canonical form of a function is the product of maxterms.
- To find the maxterms,
 - Locate all rows in the truth table where output is "0".
 - To generate the individual maxterms, substitute variables (for example A, B , or C) for zeros (of the inputs) and complements of variables (A', B' , or C') for ones in the truth table.
- Each Boolean function has a unique 2nd canonical (standard) form.

Example:

"False" (0) generating combinations: $F = 000 \quad 010 \quad 100$
 Product of maxterms: $F = (A + B + C) (A + B' + C) (A' + B + C)$

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

The 2nd canonical form of the complement of the function ($\bar{F}$) can be obtained similarly by considering the "true" (1) rows:

$$F' = (A + B + C') (A + B' + C') (A' + B + C') (A' + B' + C) (A' + B' + C')$$

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

<http://www.faculty.itu.edu.tr/buzluca>
<http://www.buzluca.info>



2011-2014 Feza BUZLUCA

2.33

Indexing maxterms:

We assign each maxterm an index (number) based on binary encoding of the variables. For example, we assign the index 5 to the maxterm $a'+b+c'$ (101) and denote that maxterm as M5.

Canonical forms are usually not the simplest (optimal) expression of the function. They can be usually simplified.

A	B	C	maxterms	
0	0	0	$A+B+C$	M0
0	0	1	$A+B+C'$	M1
0	1	0	$A+B'+C$	M2
0	1	1	$A+B'+C'$	M3
1	0	0	$A'+B+C$	M4
1	0	1	$A'+B+C'$	M5
1	1	0	$A'+B'+C$	M6
1	1	1	$A'+B'+C'$	M7

Representation of maxterms
with 3 variables

Example: 2nd canonical form of F:

$$\begin{aligned} F(A, B, C) &= \Pi M(0, 2, 4) \\ &= M0 \cdot M2 \cdot M4 \\ &= (A + B + C) (A + B' + C) (A' + B + C) \end{aligned}$$

$$F = \Pi_{A,B,C}(0, 2, 4) \text{ product of maxterms.}$$

Minimization:

$$\begin{aligned} F(A, B, C) &= (A+B+C) (A+B'+C) (A'+B+C) \\ &= (A+C)(B+B') (A'+B+C) \\ &= (A+C) (A'+B+C) \\ &= (A+C) (A'+B+C) (B+C) \text{ (consensus)} \\ &= (A + C) (B + C) \end{aligned}$$

<http://www.faculty.itu.edu.tr/buzluca>
<http://www.buzluca.info>



2011-2014 Feza BUZLUCA

2.34

Conversions Between Canonical Forms

- From 1st (sum of minterms) form to 2nd (product to maxterms) form
 - Replace the minterms with maxterms, and assign them numbers of minterms that don't appear in the 1st canonical form.
 - $F(A,B,C) = \Sigma m(1,3,5,6,7) = \Pi M(0,2,4)$
- From 2nd (product to maxterms) form to 1st (sum of minterms) form
 - Replace the maxterms with minterms, and assign them numbers of maxterms that don't appear in the 2nd canonical form.
 - $F(A,B,C) = \Pi M(0,2,4) = \Sigma m(1,3,5,6,7)$
- Finding the complement of the function in sum of minterms form
 - Select the minterms that don't appear in the 1st canonical form.
 - $F(A,B,C) = \Sigma m(1,3,5,6,7) \quad F'(A,B,C) = \Sigma m(0,2,4)$
- Finding the complement of the function in product of maxterms form
 - Select the maxterms that don't appear in the 2nd canonical form.
 - $F(A,B,C) = \Pi M(0,2,4) \quad F'(A,B,C) = \Pi M(1,3,5,6,7)$

Canonical Forms and the De Morgan Theorem

Applying the De Morgan theorem to the complement of the function in the 1st canonical form generates the expression in the 2nd canonical form.

- Complement of the function in SOP form

$$F' = A'B'C' + A'BC' + AB'C'$$
 - De Morgan

$$(F')' = (A'B'C' + A'BC' + AB'C')'$$

$$F = (A + B + C)(A + B' + C)(A' + B + C) \quad \text{2nd canonical form is obtained.}$$

Applying the De Morgan theorem to the complement of the function in the 2nd canonical form generates the expression in the 1st canonical form.

- Complement of the function in POS form

$$F' = (A + B + C')(A + B' + C')(A' + B + C')(A' + B' + C)(A' + B' + C')$$
 - De Morgan

$$(F')' = ((A + B + C')(A + B' + C')(A' + B + C')(A' + B' + C)(A' + B' + C'))'$$

$$F = A'B'C + A'BC + AB'C + ABC' + ABC \quad \text{1st canonical form is obtained.}$$